SISTEMAS DISTRIBUÍDOS — TRABALHO PRÁTICO 2

Transferência de arquivos Peer-to-Peer em C++17

Instituição: CEFET-MG Disciplina: Sistemas Distribuídos Período: 2026/2

Docente: Michelle Hanne Discente(s): [PREENCHER]

Resumo

Este trabalho apresenta a implementação de um sistema elementar de transferência de arquivos no modelo Peer-to-Peer (P2P), utilizando C++17, sockets TCP e programação concorrente no Linux. Cada processo executa simultaneamente um servidor de blocos e clientes de download. O arquivo é fragmentado em blocos de tamanho configurável, identificados por índices e validados individualmente com SHA-256. Um Peer que recebe um bloco passa a disponibilizá-lo aos vizinhos antes da conclusão do arquivo. Foram executados oito cenários iniciais com dois e quatro Peers, arquivos entre 10 KB e 20 MB e blocos de 1024 e 4096 bytes; todos passaram na verificação de integridade.

1. Introdução e objetivos

Na arquitetura P2P, os participantes podem tanto consumir quanto fornecer recursos. Diferentemente de um modelo cliente-servidor centralizado, a transferência pode ocorrer entre vários pares, distribuindo a oferta de blocos. O objetivo deste projeto é demonstrar a simetria dos Peers, a fragmentação e remontagem de arquivos e a comunicação concorrente e bidirecional entre processos.

O escopo segue o enunciado do Trabalho Prático 2: no mínimo dois Peers, vizinhos definidos estaticamente, transferência de blocos, disponibilidade imediata dos blocos recebidos, registro local de posse, verificação da integridade e avaliação de diferentes tamanhos de arquivos e blocos. O uso de tracker é opcional e foi dispensado.

2. Ambiente e ferramentas

A solução foi desenvolvida para Linux com C++17, API POSIX de sockets, std::thread, std::mutex, std::vector, chamadas pread/pwrite e biblioteca OpenSSL para SHA-256. A compilação utiliza g++ e Makefile. O protocolo opera sobre TCP/IPv4, em conexões de curta duração para cada solicitação.

Repositório do código-fonte: [INSERIR URL DO GITHUB/GITLAB ANTES DA ENTREGA].

3. Arquitetura e organização do sistema

3.1. Modelo simétrico

Cada Peer é um processo com porta própria, servidor TCP ativo e uma thread cliente para cada vizinho estático. O Seeder inicial possui o arquivo completo; os Leechers obtêm os metadados e solicitam blocos. Um bloco só é anunciado como disponível depois da escrita local e da validação de seu SHA-256. Assim, Leechers parcialmente abastecidos também atuam como fontes de blocos.

Exemplo de topologia: A (Seeder) → B e C (Leechers); D consulta B e C. As arestas representam vizinhança configurada por IP:porta, e não um tracker central.

3.2. Concorrência e consistência

O servidor utiliza select() para verificar novas conexões e cria uma thread por conexão aceita. Em paralelo, threads de download consultam periodicamente a disponibilidade dos vizinhos. Um mutex protege os vetores have (blocos armazenados) e claimed (blocos reservados para download), evitando solicitações duplicadas no mesmo Peer. O acesso ao arquivo usa pread e pwrite com deslocamento explícito, permitindo operações em blocos diferentes.

3.3. Fragmentação e armazenamento

Para um arquivo de tamanho F e blocos de B bytes, a quantidade de blocos é N = teto(F/B). O último bloco pode ter tamanho inferior a B. O Seeder calcula o SHA-256 do arquivo e de cada bloco. O Leecher cria um arquivo temporário .part, grava os blocos nas posições correspondentes e mantém um mapa de disponibilidade. Ao concluir, compara o hash integral e renomeia o arquivo temporário.

3.4. Metadados e configuração

O formato textual P2PMETA1 contém: identificação de formato, nome do arquivo, tamanho total em bytes, tamanho do bloco, SHA-256 integral e uma linha de SHA-256 para cada bloco. O Seeder grava o metadado localmente; Leechers solicitam os metadados aos vizinhos por TCP. Os endereços de vizinhos são fornecidos pela linha de comando e não são descobertos automaticamente.

Exemplo de execução: ./peer --port 5001 --neighbor 127.0.0.1:5000 --dir /tmp/p2p-b

4. Protocolo de comunicação e implementação

4.1. Mensagens de aplicação

O protocolo binário foi criado para este projeto. Cada conexão transmite um comando de um byte e recebe uma resposta cujo comprimento é explicitado em um inteiro unsigned de 32 bits na ordem de bytes da rede (network byte order). Isso evita assumir que uma chamada recv corresponde a uma mensagem completa.

Comando	Solicitação	Resposta
M — metadados	1 byte M	uint32 tamanho + texto P2PMETA1
H — disponibilidade	1 byte H	uint32 N + N bytes (0 ou 1)
G — obter bloco	1 byte G + uint32 índice	uint32 tamanho + bytes do bloco

O tamanho zero indica conteúdo temporariamente indisponível ou erro na solicitação. As rotinas sendall/recvall repetem send/recv até concluir a quantidade de bytes esperada ou detectar falha. A transferência é realizada em conexões TCP independentes por comando.

4.2. Algoritmo de download

Cada thread cliente consulta o mapa de blocos de seu vizinho. Para cada bloco que o vizinho possui, o Peer reserva o índice se ainda não o possui nem o está baixando. Em seguida solicita os bytes, verifica o hash do bloco, grava os dados na posição correta e publica sua disponibilidade. Em caso de falha, libera a reserva para nova tentativa. Quando todos os blocos são recebidos, executa a validação final.

4.3. Disponibilidade após o download

Para evitar que os Peers intermediários encerrem imediatamente e impeçam outros de terminar o download, a implementação mantém o servidor do Leecher disponível por um período adicional configurável (parâmetro --linger; padrão: 30 segundos). O Seeder permanece disponível até receber SIGINT/SIGTERM.

4.4. Limitações e decisões

A implementação suporta um arquivo por processo, IPv4 numérico e vizinhança estática. Não implementa autenticação, TLS, tracker, recuperação de blocos entre reinícios nem gerenciamento de múltiplos arquivos. O servidor é concorrente por threads, mas não é inteiramente não bloqueante. Esses limites foram aceitos para priorizar os requisitos essenciais do exercício.

5. Planejamento e execução dos testes

5.1. Parâmetros definidos no enunciado

Fator	Configuração base	Variação
Peers	2	4
Bloco	1024 bytes	4096 bytes
Arquivo pequeno	10 KB	20 KB
Arquivo médio	1 MB	5 MB
Arquivo grande	10 MB	20 MB
Vizinhos	Estáticos	Sem tracker

5.2. Procedimento

O script scripts/testes.py gera arquivos com conteúdo aleatório por os.urandom, inicia processos Peer em portas distintas na interface loopback (127.0.0.1), aguarda a criação dos arquivos recebidos e compara tamanho e SHA-256 com o arquivo original. O tempo é medido com relógio monotônico desde o início da espera até a detecção da conclusão de todos os destinos do cenário. O script produz resultados.csv.

Os testes com quatro Peers usam um Seeder A, Leechers B e C com acesso a A e D conectado a B e C. O servidor de B e C continua ativo após a conclusão para possibilitar a redistribuição. Logs de cada processo registram o índice de bloco, origem e progresso.

5.3. Critérios de aceitação

Uma execução é considerada aprovada se cada destino gerar um arquivo com tamanho exatamente igual ao original e SHA-256 idêntico. O script também verifica a conclusão dentro do prazo máximo definido para o teste. O sucesso em localhost não equivale à validação em computadores distintos: latência, congestionamento e perdas de rede podem alterar os resultados.

5.4. Reprodutibilidade

Para reproduzir: instalar g++, make e libssl-dev; executar make; em seguida python3 scripts/testes.py. As medições são sensíveis à carga do sistema, ao armazenamento e à concorrência. Para uma comparação de desempenho conclusiva, recomenda-se repetir cada cenário várias vezes e relatar média e dispersão.

6. Resultados iniciais e conclusão

6.1. Resultados de integração

Peers	Bloco (B)	Arquivo (bytes)	Tempo (s)	SHA-256
2	1024	10240	0.151	OK
2	4096	20480	0.151	OK
2	1024	1048576	0.151	OK
2	4096	5242880	0.251	OK
2	1024	10485760	0.854	OK
2	4096	20971520	0.552	OK
4	1024	1048576	0.66	OK
4	4096	5242880	0.653	OK

Fonte: execução inicial do script automatizado no ambiente de desenvolvimento, em 25/10/2026, usando 127.0.0.1. Os tempos representam uma execução por cenário, não médias estatísticas. A tabela não mede diretamente o tempo de cada conexão, apenas o intervalo de espera pelo resultado completo.

6.2. Discussão

Todos os oito cenários executados produziram arquivos com SHA-256 e tamanho corretos, confirmando a fragmentação, transmissão, validação por bloco e remontagem nos casos testados. Os cenários com quatro Peers também concluíram com sucesso, demonstrando o funcionamento da topologia configurada. Contudo, os resultados não permitem afirmar que o aumento de Peers melhora o desempenho: os arquivos, a carga e o número de blocos diferem entre algumas configurações. É necessário comparar cenários controlados e repetir as medições.

7. Conclusão

O sistema implementado atende às funções centrais de uma transferência P2P simétrica: um Peer pode servir blocos enquanto solicita outros, registrar a disponibilidade local e reconstruir o arquivo a partir de partes recebidas. A integridade foi confirmada em oito cenários iniciais. Como trabalhos futuros, destacam-se persistência de estado para retomada, distribuição inteligente de blocos, segurança das conexões, descoberta automática de vizinhos e testes em rede física com repetição estatística.

Referências e disponibilidade

CEFET-MG. Sistemas Distribuídos — Trabalho Prático 2: Transferência de Arquivos Peer-to-Peer. Michelle Hanne, 2026/2.

IEEE/POSIX. Interfaces de sockets, threads e operações de arquivo. Documentação de referência do ambiente Linux.

OpenSSL Project. Documentação da biblioteca criptográfica e da função SHA-256.

URL do repositório: [PREENCHER ANTES DA ENTREGA].